

PYTHON PROFESSIONAL DEVELOPMENT STANDARD

1. PURPOSE OF THIS DOCUMENT

This document defines **how Python code must be written in a professional environment.**

It answers:

- How should Python files be structured?
- How should functions and classes be written?
- How should errors be handled?
- How should code scale from intern-level to senior-level?
- What separates *script-level Python* from *professional Python*?

Any Python code **not following this document is considered non-professional.**

2. CORE PHILOSOPHY (VERY IMPORTANT)

A professional Python developer writes code that is:

1. **Readable** (first priority)
2. **Predictable**
3. **Testable**
4. **Maintainable**
5. **Scalable**

6. Explicit over clever

- ✗ Writing “smart” code
 - ✓ Writing “clear” code
-

3. PYTHON VERSION STANDARD

- Minimum Python version: **3.10**
 - Recommended: **3.11+**
 - Never write code assuming older versions
-

4. FILE & FOLDER STRUCTURE STANDARD

4.1 Rule: One Responsibility per File

✗ Bad

- `utils.py` # contains DB, auth, helpers, validations

✓ Good

- `db.py`
 - `auth.py`
 - `validators.py`
-

4.2 Folder Naming Rules

- Use `snake_case`

- Folder names must represent responsibility
 - `services/`
 - `repositories/`
 - `schemas/`
 - `utils/`
 - `core/`
-

5. NAMING CONVENTIONS (STRICT)

5.1 Variables

- `user_count = 10` # 
- `uc = 10` # 

Rules:

- Meaningful names
 - No single-letter names (except loop indexes)
-

5.2 Functions

- `def calculate_total_amount():` # 
- `def calcAmt():` # 

Rules:

- Verb-based
- `snake_case`

- Describe **what** the function does
-

5.3 Classes

- `class UserService:` # 
- `class user_service:` # 

Rules:

- PascalCase
 - Represents a **concept or role**
-

5.4 Constants

- `MAX_RETRY_COUNT = 3`
 - `DEFAULT_TIMEOUT_SECONDS = 30`
-

6. FUNCTION DESIGN STANDARD (CRITICAL)

6.1 Function Size Rule

- Ideal: **5–20 lines**
- Max allowed: **30 lines**

If longer → split the function.

6.2 Function Responsibility

✗ Bad

- `def create_user():`
- `validate_input()`
- `save_to_db()`
- `send_email()`
- `log_activity()`

✓ Good

- `def create_user():`
- `user = validate_user_input()`
- `save_user(user)`
- `notify_user(user)`

Each function should do **ONE logical thing**.

6.3 Function Arguments

✗ Bad

- `def create_user(name, email, phone, age, role, status):`

✓ Good

- `def create_user(user_data: UserCreateSchema):`
-

7. TYPE HINTING (MANDATORY)

All production Python **must use type hints**.

- `def get_user(user_id: int) -> User:`

- ...

Benefits:

- Self-documenting
 - IDE support
 - Safer refactoring
 - Better AI assistance
-

8. DOCSTRINGS STANDARD

8.1 Function Docstring

- `def calculate_tax(amount: float) -> float:`
- `"""`
- `Calculate tax for a given amount.`
- `"""`
- `Args:`
- `amount (float): Base amount`
- `"""`
- `Returns:`
- `float: Calculated tax`
- `"""`

Docstrings are **mandatory for public functions**.

9. ERROR HANDLING STANDARD

9.1 Never Use Bare Exceptions

✖ Bad

- `except:`
- `pass`

✔ Good

- `except ValueError as exc:`
 - `raise InvalidInputError(str(exc))`
-

9.2 Fail Fast Principle

- Validate inputs early
 - Raise clear errors
 - Do not hide failures
-

10. LOGGING STANDARD

✖ Bad

- `print("User created")`

✔ Good

- `logger.info("User created", extra={"user_id": user.id})`

Rules:

- No `print()` in production

- Use logging levels: DEBUG, INFO, WARNING, ERROR, CRITICAL
-

11. COMMENTS STANDARD

11.1 Good Comments Explain *Why*, Not *What*

 Bad

- `i += 1 # increment i`

 Good

- `# Increment retry count to avoid infinite loop`
 - `retry_count += 1`
-

12. OBJECT-ORIENTED PYTHON (PRO LEVEL)

12.1 When to Use Classes

Use classes when:

- State must be preserved
 - Logic belongs to a domain concept
 - Dependency injection is needed
-

12.2 Avoid God Classes

 Bad

- `class UserManager:`

- `# auth, db, validation, notification`

✓ Good

- `class UserService:`
 - `class UserRepository:`
 - `class Validator:`
-

13. DATA VALIDATION STANDARD

- Never trust external input
 - Validate at system boundaries
 - Use schemas / validators
-

14. TESTING MINDSET (VERY IMPORTANT)

A professional developer writes code **assuming it will be tested**.

Rules:

- Functions should be testable
 - No hard-coded values
 - No hidden dependencies
-

15. PERFORMANCE & CLEAN CODE RULES

- Prefer readability over micro-optimizations
 - Use list comprehensions **only if readable**
 - Avoid deeply nested logic (max 2 levels)
-

16. SECURITY BASICS EVERY PYTHON DEV MUST KNOW

- Never hardcode secrets
- Never log passwords
- Validate inputs
- Sanitize user data
-